

Q.1.

Process

A process is a program in execution.

- A program is just a passive set of instructions stored in memory.
- A process is an active entity that includes:
 1. Program code
 2. Current activity
 3. Process stack
 4. Data section.

So, when we run a program, the OS creates a process.

Component of process :-

1. Text section → Actual program code.
2. Program Counter (PC) → tells which instruction to execute next.
3. Stack → Stores temporary data like function calls, return values, local variables.
4. Data section → Contains global variable.
5. Heap → dynamic memory.

Process Control block (PCB) :-

The OS keeps information about each process in a structure called PCB.

It contains:-

- Process ID (PID)
- Process state (new, ready, running, waiting, terminated)
- CPU register, PC
- Scheduling info (priority, queues)
- Memory info (base/limit registers, page table)
- I/O status.

Q. 2.

Process States

~~Define~~ A process state is the current condition of a process, which shows what the process is doing at a particular time.

It represents the status of the process in its lifecycle, such as whether it is being created, waiting for CPU, actively executing, waiting for I/O or finished.

Main process states:-

1.) New :-

- Process is being ~~executed~~ created.

2.) Ready :-

- Process is loaded into main memory and waiting for CPU.
- It is ready to run.
- Stored in Ready Queue.

3.) Running :-

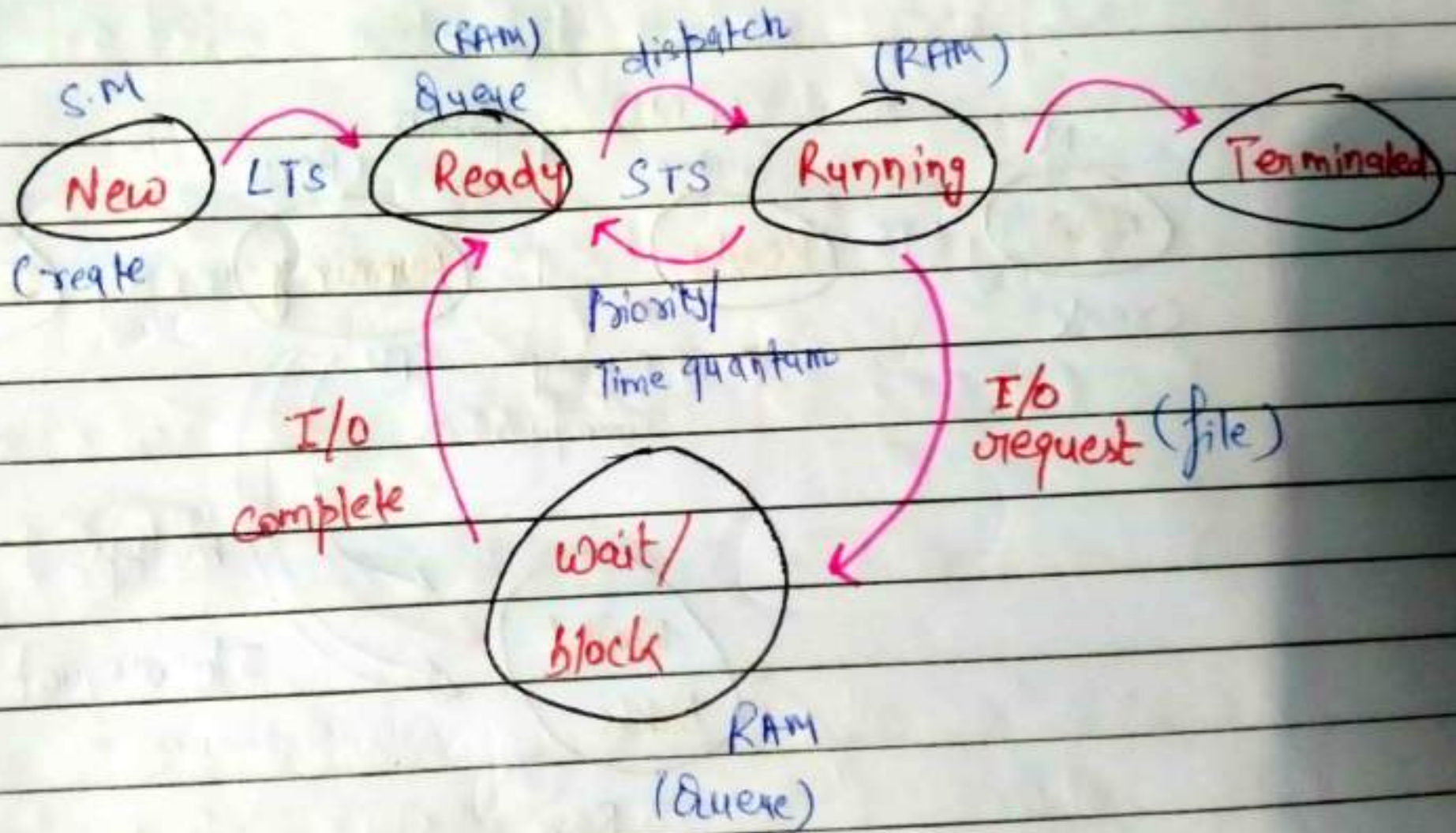
- Process is executing on the CPU.
- Only one process per CPU core can be in this state at a time.

4.) Waiting (or blocked) :-

- Process cannot continue until an event occurs (I/O completion, resource release).
- CPU is not used while waiting.
- Stored in wait queue.

5.) Terminated (or Exit) :-

- Process has finished execution.



Q. 2. User mode v/s Kernel mode.

The CPU in modern Computer works in two modes:

- **User mode** → where normal programs run.
- **Kernel mode** → where the operating system runs.

This separation ensures security & stability of the system.

1.) User mode:-

The mode in which application program runs.

- program cannot directly access hardware (disk, I/O, memory, CPU instr.).
- If a program needs hardware access, it must ask the OS via system call.

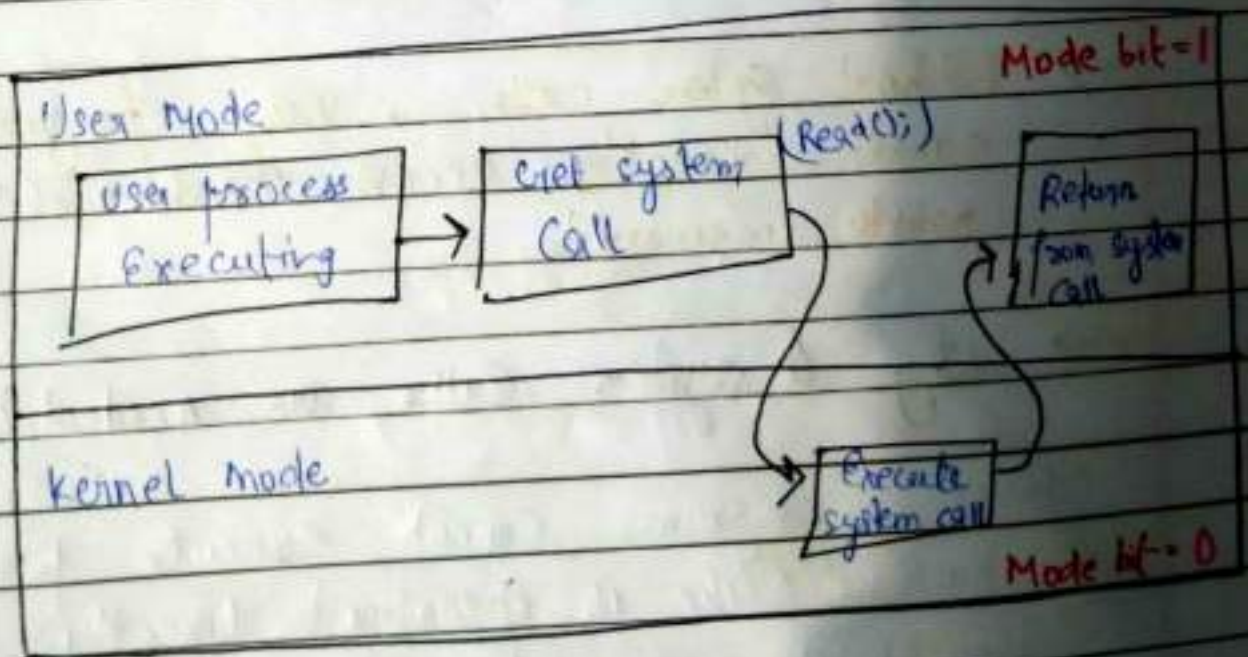
Eg: When we open MS word or run a C program, it runs in user mode.

2.) Kernel mode:-

The mode in which the operating system runs.

- Has full access to **hardware and system resources.**
- Executes privileged instructions (I/O operation, memory management).
- Handles **system call, interrupts, device drivers, scheduling, memory allocation.**

Eg: when your program does a **printf()**, it internally makes a system call to the OS, which runs in kernel mode to send data to the screen.



System Call

A system call is the ~~interface~~ ^{inter}face between a user program and the operating system (kernel).

It allows a program running in user mode to request a service from the OS kernel.

→ Without system calls, a user program cannot directly access hardware (disk, printer, memory).

→ Why system calls are needed?

- User program cannot execute privileged instructions (like IO operation) directly.
- They must request the OS to do it safely and securely.

Steps in a system call:-

- 1.) Program request service (eg: read a file).
- 2.) It call a library function (eg: `read()` in C).

3. Library function makes a trap (software interrupt) to switch user mode $\rightarrow$ kernel mode.
4. OS kernel performs the request task (eg. fetch data from disk).
5. Control is returned to the user program (kernel $\rightarrow$ User mod).

Q.5.

Threads

A thread is the smallest unit of execution inside a process.

- A process is like a program in **execution**.
- A process can have one or more **threads**.
- All threads of the same process **share the same memory** (code, data, files), but each thread has its own stack, program counter, and register.

Note → Think of a process as a house, and threads as people living in the same house.

They share as the same kitchen (memory, files), but each person has their own room (stack, register).

Why use threads?

1. **Faster execution** - Creating a thread is lighter than creating a new process.
2. **Parallelism** - multiple threads can run at the same time on multiple CPUs.
3. **Better resource sharing** - Since threads of the same process share memory, communication is faster than between processes.

4. Responsive programs - Eg. in a browser:

- One thread load a page
- Another plays video.
- Another listen for user input.

Process	Threads
1.) System call involved in process.	1.) There is no system call involved in process . (in process).
2.) OS treats different process differently	2.) All user level threads treated as single task for OS
3.) Different process have different copies of data, files, code.	3.) Threads share same copy of code and data.
4.) Context switching is slower.	4.) Context switching is faster.
5.) Blocking a process will not block other.	5.) Blocking a thread will block entire process.

Types of thread :-

User level thread	Kernel level thread
1.) User level threads are managed by User level library	1.) Kernel level threads are managed by OS system calls.
2.) User level threads are typically fast	2.) Kernel level threads are slower than User level.
3.) Context switching is faster.	3.) Context switching is slower.
4.) If one user level thread is performing blocking operation then entire process gets blocked.	4.) If one kernel level thread is blocked, it has no effect on others.

Q.6. First Come First Serve (FCFS) :-

- The simplest CPU scheduling algorithm.
- Processes are executed in the **order they arrived** in the ready queue.
- It works just like a queue in a shop $\rightarrow$ first customer served first.

Characteristics :-

- Type $\rightarrow$ **Non-preemptive** (once a process gets CPU, it runs until completion).

~~Basis of Arrival time (AT)~~

Basis $\rightarrow$ Arrival time [depend on AT] [from 0 to increasing order]

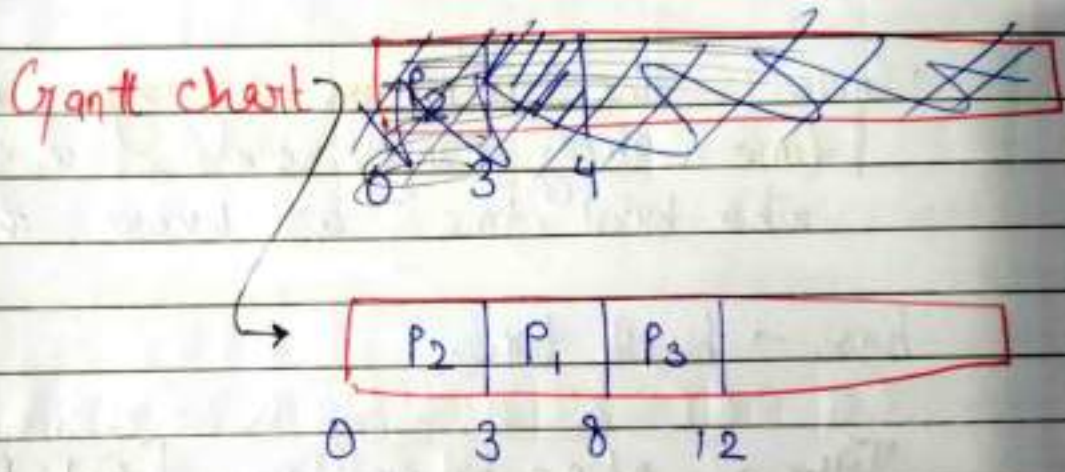
- here, **$RT = WT$ [Non-preemptive]**.

- No starvation { if new high-priority processes keep arriving, a low priority process may never get CPU }.

Note:- If arrival time is same for two or more processes then pick which is written first because fcfs (queue)

Eg:

Process	BT	AT	CT	TAT = CT - AT	WT = TAT - BT
P ₁	5	2	8	6	1
P ₂	3	0	3	3	0
P ₃	4	4	12	8	4



Avg. TAT = $\frac{6+3+8}{3} = \frac{17}{3} = 5.66$

Avg. WT = $\frac{1+0+4}{3} = \frac{5}{3} = 1.66$

Q.7. Shortest Job first (SJF):-

Shortest job first (SJF) schedules the process with the smallest CPU burst time next.

It aims to minimize average waiting time.

Characteristics:-

- Gives the minimum average waiting time among non-preemptive algorithm when burst time are known in advance.

basis $\rightarrow$ burst time.

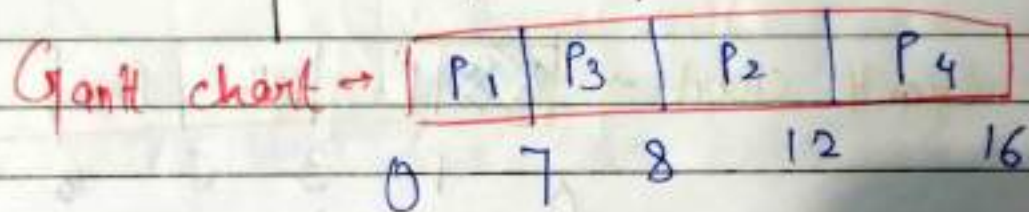
Type $\rightarrow$ Non-preemptive. $[WT = RT]$

Note:-

- ① If same Arrival time of two or more processes then, whose burst time is less will come at first.
- ② If two process have same burst time then, who came earlier with Arrival time ~~(who is written first in queue)~~ will come (use fcts).

Eg:

Processes	AT	BT	CT	TAT = CT - AT	WT = TAT - BT	RT = FT - AT
P ₁	0	7	7	7	0	0
P ₂	2	4	12	10	6	6
P ₃	4	1	8	4	3	3
P ₄	5	4	16	11	7	7



$$\text{Avg TAT} = \frac{7+10+4+11}{4} = \frac{32}{4} = 8$$

$$\text{Avg WT} = \frac{0+6+3+7}{4} = \frac{16}{4} = 4$$

Q.8. • Shortest Remaining Time First (SRTF) :-
(SJF with preemption)

• At any instant the CPU is assigned to the process that has the smallest remaining CPU burst.

• If a new process arrives with remaining time less than the current running process's remaining time, the running process is preempted.

Characteristics:-

Type $\rightarrow$ Preemptive ($WT \neq RT$)

basis $\rightarrow$ burst time

Same as SJF but different is, it is preemptive.

Note:-

Gantt chart $\rightarrow$

P ₁	P ₂	P ₂	P ₂	P ₃	P ₃	P ₃	P ₄	
0	1	2	3	4	5	6	7	8

here, $CT(P_2) \rightarrow 4$, $FR = 1$

$CT(P_3) \rightarrow 7$, $FR = 4$

Eg:

Processes	AT	BT	CT	TAT CT-AT	WT TAT-BT	RT FR-AT
P ₁	0	85	9	9	3	0
P ₂	1	3210	4	3	0	0
P ₃	2	7	16	14	7	7

Grant chart →

P ₁	P ₂	P ₂	P ₂	P ₁	P ₁	P ₁	P ₁	P ₁	P ₃
0	1	2	3	4	5	6	7	8	9

$$\text{Avg. TAT} = \frac{9+3+14}{3} = \frac{26}{3} = 8.66$$

$$\text{Avg. WT} = \frac{3+0+7}{3} = \frac{10}{3} = 3.33$$

$$\text{Avg. RT} = \frac{0+0+7}{3} = \frac{7}{3} = 2.33$$

Q.9.

• Round Robin :-

Type:- Preemptive CPU scheduling.

Mechanism:- Each process is assigned a fixed ~~time~~ time quantum (q) or time slice.

It means After q unit of time, a timer interrupt occurs $\rightarrow$ The OS perform a context switching if the process hasn't finished.

• It has two queue (Ready & Running).

• The CPU executes a process for at most q unit of time, then preempts it and move it to the end of the ready queue (if not finished).

Note:- If in question time quantum = 2 unit and if a process takes 1 unit then at that time in Gantt chart it will take 1 unit of that process and for next process it will take 2 unit of time.

Eg)

Gantt chart $\rightarrow$

P ₁	P ₂	P ₃	P ₁	P ₁	P ₂
----------------	----------------	----------------	----------------	----------------	----------------

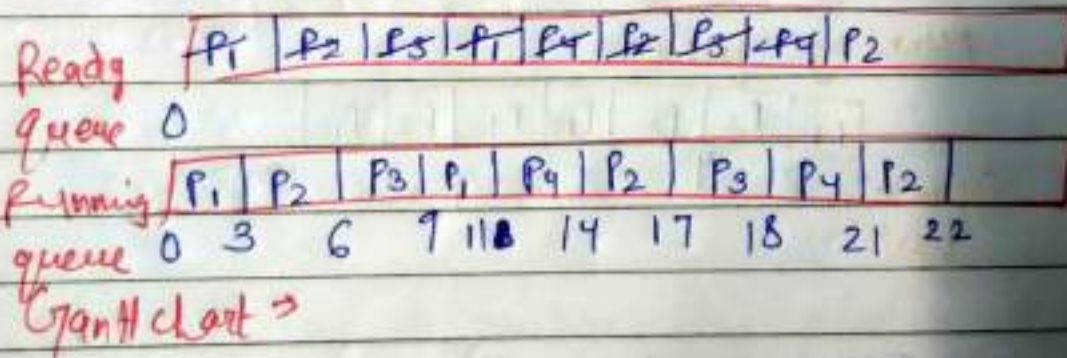
0 2 4 6 8 9 11

T.q = 2 unit

Eg:

Processes	AT	BT	CT	TAT CT-AT	WT AT-BT	RT FR-AT
✓ P1	0	8	11	11	6	0
✓ P2	1	7	22	21	14	2
✓ P3	3	4	18	15	11	3
✓ P4	5	6	21	16	10	6

Time quantum = 3 units



$$\text{Avg TAT} = \frac{11 + 21 + 15 + 16}{4} = \frac{63}{4} = 15.75$$

$$\text{Avg WT} = \frac{6 + 14 + 11 + 10}{4} = \frac{41}{4} = 10.25$$

$$\text{Avg RT} = \frac{0 + 2 + 3 + 6}{4} = \frac{11}{4} = 2.75$$

Q. 10.

• Preemptive Priority CPU scheduling :-

- Each process is assigned a priority number.
- **Smaller number = higher priority** (unless question states otherwise).
- The CPU is allocated to the **highest priority processes**.
- If a new process arrives with higher priority than the currently running one, it preempts (interrupts) the running process.

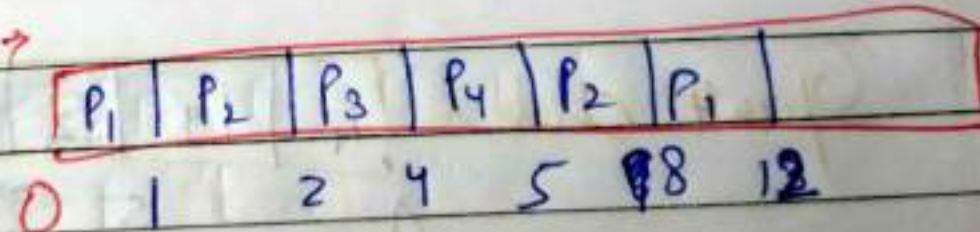
~~Not~~ Basis → Priority.

- Same as SRTF just it has priority.

Eg: Higher the number, higher the priority,
 final Avg WT

Process	AT	BT	Priority	CT	TAT CF+AT	WT TAT-BT
P ₁	0	5	10	12	12	7
P ₂	1	3	20	8	7	4
P₃	2	2	30	4	2	0
P₄	4	1	40	5	1	0

Gantt chart →



$$\text{Avg WT} = \frac{7+4+0+0}{4} = 11/4 = 2.75$$

Q.11.

fork() system call

fork() is a system call in Unix/Linux used to create a new process.

- The new process created is called the **child process**.
- The original process is called the **parent process**.
- Both process will execute the code after the **fork()** call.

→ How fork() works:-

1.) when a process calls fork() :

- The OS makes a **copy of the parent processes**.
- Both parent and child have the same code, same data, same stack, same heap.
- They only differ in process ID (PID).

2.) After `fork()` returns :-

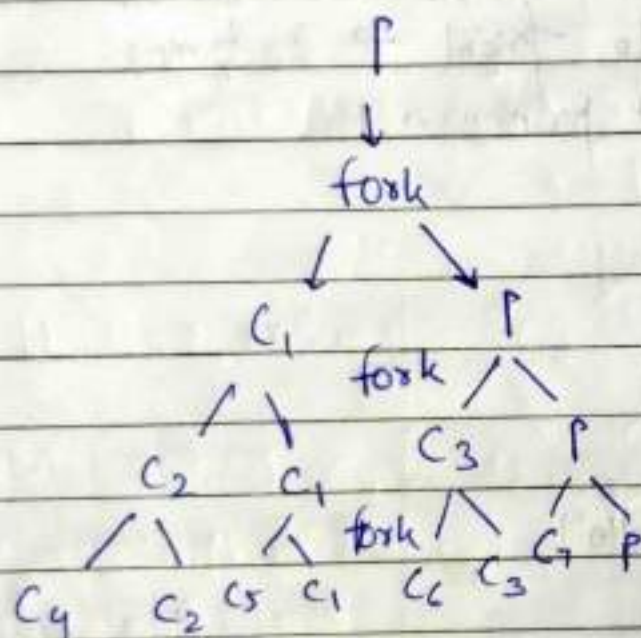
- In the present process, `fork()` return the child's PID (positive number).
- In the child process, `fork()` returns 0.
- If `fork()` fails, it return -1 (no child created).

`fork()` → parallel execution

Eg: if we want to ^{execute} both 2 and 3 table simultaneously we use `fork()`.

Eg:

```
main()
{
    fork();
    fork();
    fork();
    printf("hello");
}
```



No. of total process = 8
" " child " = 7

Formula $\rightarrow$ Total no. of process or execution = 2^n

Total no. of child process = $2^n - 1$

Eg:

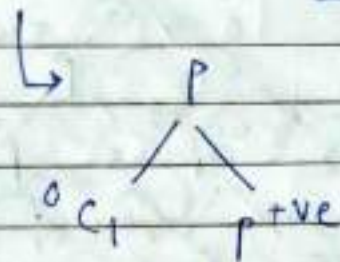
```
int main()
{
    if (fork() && fork())
    {
        fork();
        printf("hello");
        return 0;
    }
}
```

find no. of child process and total process.

if(fork())

3

4



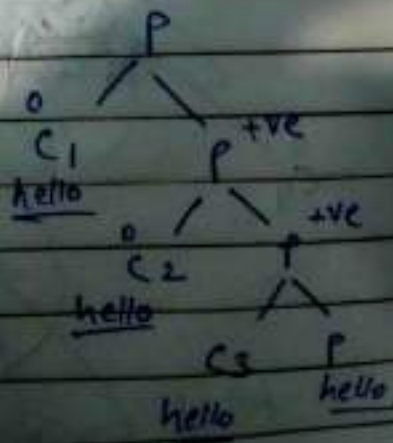
if(fork() && fork())

0 && something → 0 → No child

∴ direct(hello)

1st first → 0 then direct outside without checking second

if(fork() && fork())
fork();



~~Wrong!~~

Date: / /

Page:

Eg:

```
main()
{
```

```
    if (fork() || fork())
```

```
    { fork();
```

```
      printf("hello");
```

```
    }
```

if (fork⁰() || fork¹) fork()

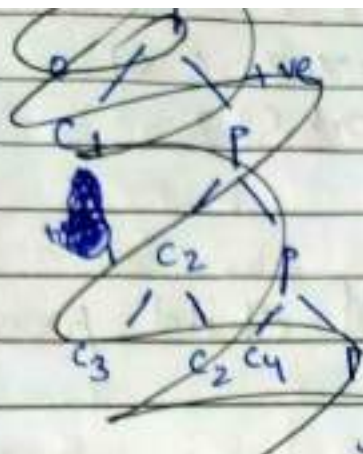
or
if (fork⁰() || fork⁰)

0 || 0 → nothing

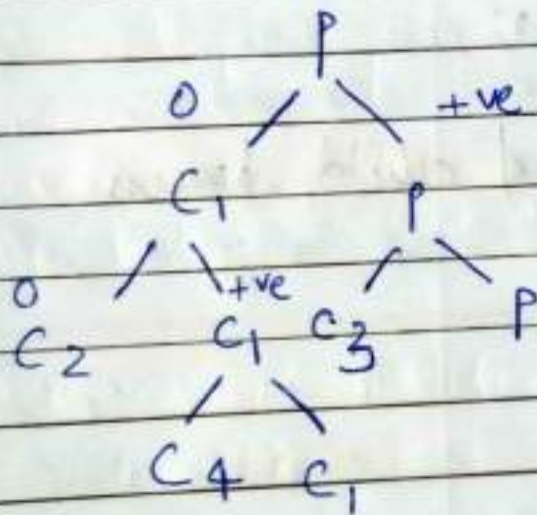
0 && something

if first 0 then it
does not go to
second in AND
but in OR it goes





No. of total execution $\rightarrow 5$
 " " child process $\rightarrow 4$



Total $\rightarrow 5$

child $\rightarrow 4$

✓

if first is 1 then it not check second it directly come outside

Q.12. Process Synchronization

Date: ___/___/___

Page: ___

Process synchronization is the coordination of processes that share resources such as memory, files or data, in a concurrent system (like a multiprogramming OS).

It ensure that only one process accesses a shared resource or critical section at a time - so the data remain constant and correct.

Critical section: A critical section is a part of the program where the process accesses shared resource.

Each processes has:- [how synchronisation works]

1. ✓ Entry section: Request to enter critical section
2. ✓ Critical section: Access shared data.
3. ✓ Exit section: Leaves critical section
4. ✓ Remainder section: Other code.

Need for synchronization:-

1. ✓ To maintain data consistency in shared memory.
2. ✓ To avoid race conditions.
3. ✓ To ensure correct execution order of processes.
4. ✓ To coordinate independent tasks (like producer-consumer).

~~On~~ On the basis of synchronization, process are categorized as one of the following two types:

- **Independent process**: The execution of one process does not affect the execution of other processes.
- **Cooperative process**: The process that can affect or be affected by other processes executing in the system.

Q.12.

Critical Section Problem

What is the critical section problem?
In an operating system, processes often share resource, like:

- A variable
- A file
- A printer
- Memory, etc.

When multiple process access and modify shared at the same time, it can lead to inconsistent result → This is the race condition we just learned.

So, we need a rule:

- Allow only one process to access the shared resource at a time.

→ That part of code (where shared data is accessed) is called the Critical section.

Definition:-

The critical section problem is to design a ~~program~~ protocol that allow processes to cooperate while sharing resource so that no two processes execute in their critical section simultaneously.

Eg;

let say two processes P_1 and P_2 both want to update a shared variable x .

// shared variable

int $x = 0$;

process P_1 ;

$x = x + 1$;

process P_2 ;

$x = x + 1$;

If both execute together (without synchronization), we can get wrong output, that is, the **critical section problem**.

So we must make sure :

- If P_1 is in critical section $\rightarrow P_2$ must wait.
- If P_2 is in its critical section $\rightarrow P_1$ must wait.

Solution Condition (3 must be satisfied)

Any correct solution to the critical section problem must satisfy these 3 conditions:

Condition	Meaning
1/ Mutual Exclusion	Only one process can execute in its critical section at a time.
2/ Progress	If no process is in its critical section, and some process wish to enter, then the selection of the next process that can enter should not be postponed indefinitely.
3/ Bounded waiting	A process should not have to wait forever to enter its critical section.

Code Structure:-

```
do
{
    // Entry section → Request to enter critical section
    // Critical section → Access shared resource
    // Exit section → Release lock/permission
    // Remainder section → Other work (not shared)
}
while (true);
```

How it solved (in OS):

Different methods are used to handle the critical section problem:

<u>Approach</u>	<u>Example</u>
Software Solution	Peterson's Algorithm, Bakery algorithm
Hardware solution	Test-and-set, swap instruction
OS-level mechanisms	Semaphores, Mutex Locks, Monitors

Real-life eg

Think of a bathroom with One key:

- Bathroom: Shared resource (critical section)
- People: processes
- key: lock (synchronization mechanism)

~~Rules~~ Rules:

1. Only one person can have the key (Mutual exclusion).
2. If the bathroom is empty, someone not waiting can enter (progress).
3. No one should wait forever (Bounded waiting).

Q.14.

Deadlock

Date _____

Page _____

Deadlock is a situation in an operation where two or more processes are waiting for each other indefinitely to release resources, and therefore, none of them can proceed.

Real-life eg;

Person	Resource holding	Resource waiting for
A	Pen	paper
B	paper	pen

- A won't release the pen until they get paper.
- B won't release the paper until they get the pen.

* Both keep waiting - forever - This is deadlock.

In OS term:

Processes	Holding	Waiting for
P ₁	Resource R ₁	Resource R ₂
P ₂	Resource R ₂	Resource R ₁

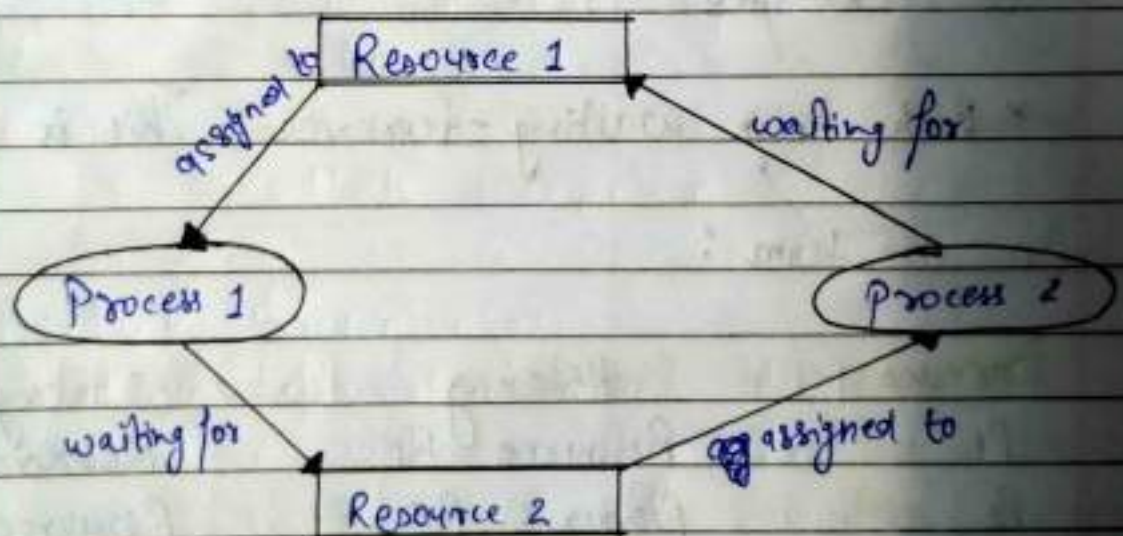
Both are blocked - neither can continue execution

→ Why deadlock Occur:

Deadlock usually happens in **resource allocation system**, When processes completes for **shared resource** (like memory, I/O device, files, ~~etc~~ etc).

Eg:

- A printer & a scanner are shared among processes.
- One process holds printer and ~~waits~~ waits for scanner.
- another holds ~~the~~ scanner and waits for printer → **Deadlock**.

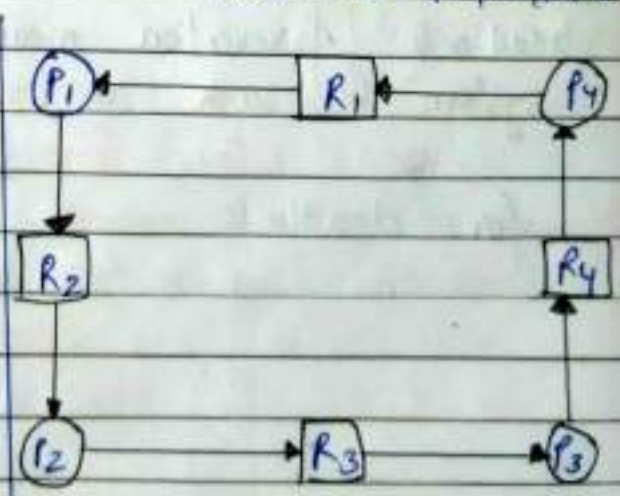


Necessary

→ Four Condition (Coffman's conditions) :-
Deadlock can occur only if all these 4 condition holds simultaneously:

No.	Condition	Meaning
1.	Mutual Exclusion	A resource can be held by only one process at a time.
2.	Hold and Wait	A process holds one resource and waits for another.
3.	No preemption	A resource cannot be forcibly taken away; it must be released voluntarily.
4.	Circular Wait	A circular chain of processes exists, where each waits for a resource held by the next. for eg; imagine four processes P1, P2, P3, P4 and four resource R1, R2, R3 and R4

- P1 is holding R1 and waiting for R2 (which is held by P2).
- P2 is holding R2 and waiting for R3 (which is held by P3).
Similarly for P3 and P4.



→ Ways to handle Deadlock:-

Q.15.

Strategy

Description

Deadlock prevention

Design system so one of the four condition ~~never~~ never holds.

Deadlock Avoidance

Dynamically check resource allocation

Deadlock Detection

Allow deadlock, then detect and recover.

Deadlock Ignorance

OS pretends it never happens (used in most system like windows, Linux).

Deadlock Prevention & Avoidance

• Deadlock prevention:-

Deadlock prevention means designing the system in such a way that at least one of the four necessary condition for deadlock never holds.

→ How to prevent each condition :-

Condition	How to prevent	Example
Mutual Exclusion	Make resource sharable if possible	Read-only files can be shared by multiple process

Holds & wait	Process must request all resource at once, before execution.	A process gets everything it needs at the start.
--------------	--	--

No preemption	If a process is waiting, take away its held resource and give them to another process	Preempt CPU, memory page etc.
---------------	---	-------------------------------

Circular wait	Impose a total ordering on resources - processes must request in increasing order only.	First ask for R ₁ , then R ₂ , then R ₃ (never reverse)
---------------	---	--

• Deadlock Avoidance :

Deadlock avoidance means the system **dynamically checks** whether granting a resource will lead to a **deadlock** or not - and **allocate only if it's safe**.

So, it doesn't prevent all four condition - instead, it **analyzes resource request** in real-time.

Key idea - "Safe State"

A safe state means there is at least one sequence of process execution such that all processes can complete without deadlock.

Deadlock Detection and Recovery

→ Deadlock detection:-

Deadlock detection means the system allows deadlocks to occur but has a mechanism to detect when they happen and then recover from them.

→ Deadlock recovery:-

(a) Process termination:-

two strategies:

1. Abort all deadlocked processes.
→ Simple but waste work.
2. Abort one at a time until deadlock ends.

→ Need to choose wisely.

factors to consider:

- Process priority
- How much it has executed
- How many resource it hold
- Whether it can be restarted easily

(b) Resource Preemption

Temporarily take resource away from some processes and give them to others.

issues:

- Which process to preempt?
- Roll back
- Starvation.

Memory management is the function of the operating system that handles the allocation and deallocation of memory to processes during their execution.

It decides:

- Which process will get memory,
- How much memory it will get, and
- When it will be taken back.

→ Why it is needed?
because:

- Main memory (RAM) is limited.
- Multiple processes need to run simultaneously.
- OS must ensure efficient use of memory and no overlap between processes.

→ Goal of memory management:-

- Efficient memory utilization: Use available RAM as much as possible without waste.
- Fast Access: Allocate and free memory quickly.
- Protection: Prevent one process from accessing another's memory.
- Relocation: Move processes within memory during execution if needed.
- Sharing: Allow some processes to share code/data safely.

→ Types of memory management schemes :-

~~Scheme~~

1. **Contiguous allocation** : Each process occupies a single contiguous block of memory.
2. **Paging** : Memory is divided into equal-sized block (page & frame).
3. **Segmentation** : Memory divides into variable-sized segments (like code, stack, data).
4. **Paging + Segmentation** : Hybrid of both - logical to physical translation via both.

Q.17. Memory Partitioning

Memory partitioning is the way the OS divides main memory into different sections (partitions) to store processes.

Each partition can hold one process at a time.

Depending on whether partition sizes are fixed or dynamic, we have two schemes:

- 1) Fixed-size partitioning
- 2) Variable-size partitioning

} Continuous process

Fixed-Size Partitioning

Main memory is divided into equal-size partitions (fixed blocks) at system startup. Each partition can hold exactly one process at a time.

Example:-

- Memory = 400 KB
- Partition size = 100 KB

Then memory is divided like this:

P1	100 KB
P2	100 KB
P3	100 KB
P4	100 KB

if :

- process A = 70 KB $\rightarrow$ occupies one 100 KB partition $\rightarrow$ 30 KB wasted inside.
- process B = 90 KB $\rightarrow$ 10 KB wasted inside.

$\rightarrow$ Wasted memory inside partition = Internal fragmentation.

Problems with fixed partitioning :-

- 1.) Internal fragmentation: Memory wasted inside partition.
- 2.) Limited multiprogramming: If few partition, only few processes fit.
- 3.) Unequal process size $\rightarrow$ large process may not fit at all.

Variable-size partitioning

Main-memory is divided dynamically into partitioning of different sizes - each created as per the size of the process.

No fixed partition - The OS creates exactly the amount A of memory needed for each process.

Eg:

Suppose:-

- Memory = 400 KB
- Process A = 120 KB
- Process B = 80 KB
- Process C = 100 KB

PA	120 KB
PB	80 KB
PC	100 KB
FREE	100 KB

No Internal fragmentation

But possible External fragmentation

Problem with variable partitioning:-

1.) External fragmentation:

Free memory gets divided into small non-contiguous holes that can't be used.

2.) Compaction Needed:

OS must occasionally rearrange processes to combine free space.

3.) Complex management:

Must keep track of variable holes.

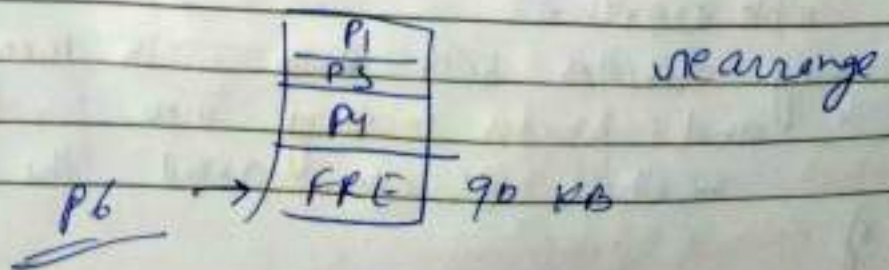
Eg:

P1	80 KB
P2	50 KB
P3	100 KB
P4	120 KB
P5	40 KB

let P2 and P5 remove then it generate
hole
And let ~~one~~ ^(P6) one more process wants to
enter of memory space ~~80 KB~~ = 80 KB

P1	80 KB
hole	50 KB
P3	100 KB
P4	120 KB
hole	40 KB

Space left = 90 KB (50 + 40)



→ Need of Paging :-
Q.18.

Earlier, OS used Contiguous memory allocation (fixed or variable partitioning).

That means:

- Each process had to be loaded in one continuous block of memory.
- So the process's code and data must fit entirely in a single chunk of RAM.

This create several serious problem:

- ① Internal fragmentation
- ② External "
- ③ Compaction overhead.

Solⁿ → Paging

→ What is paging :-

Paging is a memory management techniques that allows the physical address space (RAM) of a process to be non-contiguous.

In simple words:-

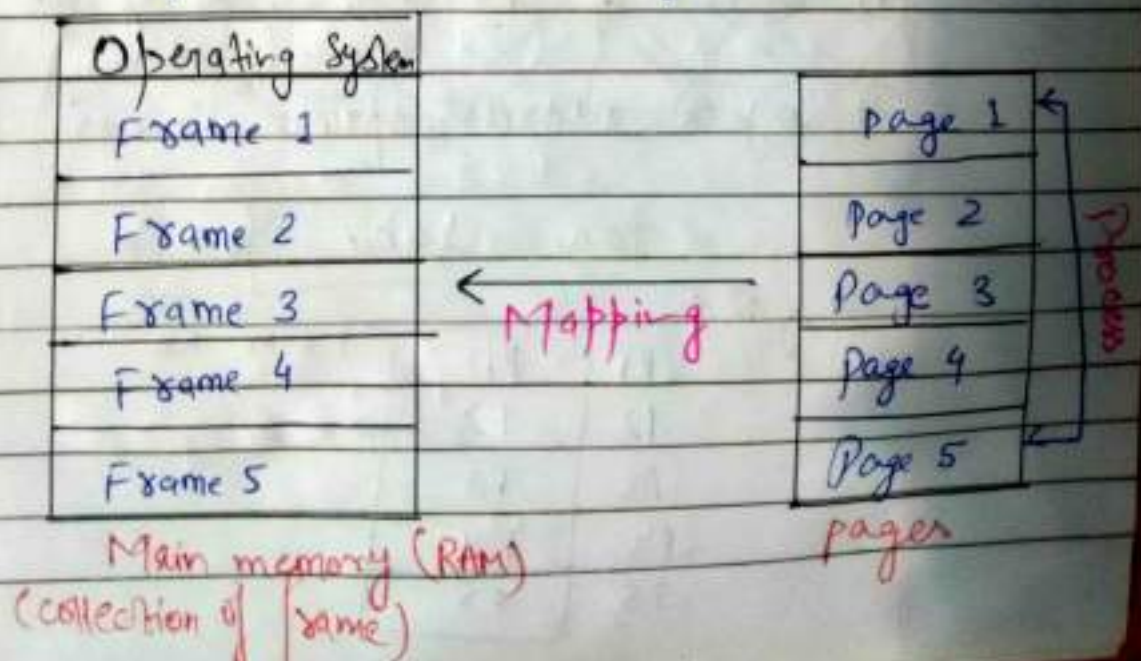
Paging divides both ~~main~~ main memory and process memory into fixed-size block so that we can avoid the problem of external fragmentation.

In other words:-

Paging is the process of moving part of a program called pages, from secondary storage (ROM) into the main memory (RAM).

The main idea behind paging is to ~~break~~ break a program into smaller size blocks called pages.

- Process does not have to be allocated in contiguous memory space.
- The whole process does not have to be in main memory, some pages can be present and some pages can be loaded when needed. It allows more processes and even processes larger than main memory to run.
- Memory allocation is simplified as memory is always allocated in fixed sized pages.



Imp

1 byte = 8 bit

Eg: 48 bit = 6 byte

Formula sheet :-

Q.19

Concept

Formula

Explanation

Logical Address
space (LAS)

2^m bytes

m = bits in
logical address

Physical address
space (PAS)

2^p bytes

p = bits in
physical address

Page size
(frame size)

2^n bytes

n = bits for offset

Number of / Number
pages / of entries
in page table

2^{m-n} pages

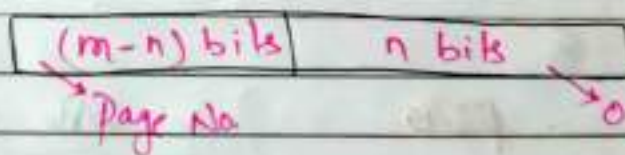
pages in logical
address space

Number of
frames

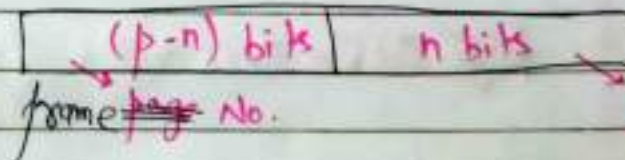
2^{p-n} frames

frame in physical
address space

Logical address
format



Physical address
format



Note:- $m, n, p, m-n, p-n$ all are in bits
 $2^m, 2^n, 2^p, 2^{m-n}, 2^{p-n}$ all are in bytes

Page Table
Size (PDS)

→ Address Translation in paging :-

When the CPU generate a logical address, it must be converted into a physical address (real address in RAM).

This translation is done by the **MMU** (memory management unit) using the page table.

Step 1:-

$$\text{Page No.} = \frac{\text{Logical address [floor value]}}{\text{page size}} \quad \text{Page table}$$

∴ The OS looks up the frame no. for that page No. in the

Step 2:- **offset** = logical address % page size

Step 3:- **Physical address** = (frame no. × frame size) + offset.

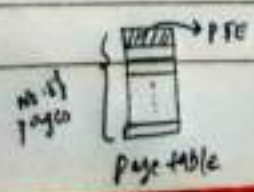


~~Page~~

Page table size (PTS) :-

~~Page table size = (No. of entries in page table) × (Size of page table entry)~~

$$\text{Page table size (PTS)} = \underbrace{(\text{No. of entries in page table})}_{\text{or (No. of page)}} \times \underbrace{(\text{Size of page table entry (PTE)})}_{\text{Unit} \rightarrow \text{byte}}$$



PTS → Total size of one page table + valid, dirty, permission bit

PAGE Table Entry format :-

Frame No.	Valid	dirty	Permission bit
-----------	-------	-------	----------------

← mandatory →
↑
PFS
frame size

0/1 0/1

If we add all value then we get PTE.
Date
Page

- Q. Consider a demand paging memory management system with 32-bit logical addresses, 20-bit physical addresses, and page size of 2048 bytes. Assume that the memory is byte addressable, what is the max^m no. of entries in the page table?

=> Given

logical address = 32-bit

1. LAS = 2^{32} bytes

Physical address = 20 bit

Page-size = 2048 bytes = 2^{11} bytes

to find: No. of page

$$\text{No of page} = \frac{\text{LAS}}{\text{page size}} = \frac{2^{32} \text{ bytes}}{2^{11} \text{ bytes}} = 2^{21}$$

~~Memory Management~~

- Q. Consider a system with byte-addressable memory 32-bit logical address, 4 kilobyte page size and page table entries of 4 bytes each. The size of the page table in the system in the megabyte is —
=) Given;

$$\text{logical address} = 32\text{-bit}$$

$$LAS = 2^{32} \text{ byte}$$

$$\begin{aligned} \text{page size} &= 4 \text{ KB} \\ &= 2^{12} \text{ Byte} \end{aligned}$$

$$\text{Page table entries (PTE)} = 4 \text{ Byte}$$

$$\text{Size of page table (PTS)} = (\text{No. of pages}) \times \text{PTE}$$

$$\begin{aligned} \text{No. of pages} &= \frac{LAS}{\text{page size}} = \frac{2^{32} \text{ Byte}}{2^{12} \text{ byte}} \\ &= 2^{20} \end{aligned}$$

$$\begin{aligned} \text{PTS} &= 2^{20} \times 4 \\ &= 2^{22} \\ &= \underline{\underline{4 \text{ MB}}} \end{aligned}$$

- Q. Consider a system with 40-bit virtual addressing and page size of 16 kilobyte. If the computer system has a one-level page table per process and each page table entry required 48 bit then the size of the per-process page table is 384 megabyte.

⇒

Given;

logical address = 40-bit

$$LAS = 2^{40} \text{ Byte}$$

$$\text{Page size} = 16 \text{ KB}$$

$$= 16 \times 2^{10} \text{ Byte}$$

$$= 2^{14} \text{ Byte}$$

$$\begin{aligned} \text{Page table entry (PTE)} &= 48 \text{ bit} \\ &= 6 \text{ byte} \end{aligned}$$

$$\text{page table size} = (\text{No. of page}) \times \text{PTE}$$

$$\begin{aligned} \text{No. of page} &= \frac{LAS}{\text{page size}} = \frac{2^{40}}{2^{14}} \\ &= 2^{26} \end{aligned}$$

$$PTS = 2^{26} \times 6$$

$$= 2^6 \times 6 \times 2^{20} = \underline{\underline{384 \text{ MB}}}$$

Memory is an address space.

Date / /

Page

PRG-2022

Q. Consider a simple paging system with the following parameters:

- 2^{32} bytes of physical memory
- page size 2^{10} byte
- 2^{16} page of logical address space.

- ① How many bits are in a logical address?
- ② How many bits are in a frame?
- ③ How many ~~bits~~ ^{bytes} in the frame physical address
Specify the frame!
- ④ How many entries in the page table?

⇒

Given

$$\text{Physical address} = 2^{32} \text{ bytes}$$

$$\text{page size} = 2^{10} \text{ bytes}$$

$$\text{No. of pages in LAS} = 2^{16}$$

① bits in logical address

$$\text{No. of page} = \frac{\text{LAS}}{\text{page size}}$$

$$\text{LAS} = (\text{No. of pages}) \times \text{page size}$$

$$= 2^{16} \times 2^{10} \text{ byte}$$

$$\text{LAS} = 2^{26} \text{ byte}$$

logical address = 26 bit

② page size = frame size

$\therefore$ byte in frame = 2^{10} byte

③ bits in physical address

Physical address space = 2^{32} byte

~~frame size = 2^{10} byte~~

~~frame size = 2^{10} byte~~

bits in physical address = 32

④ No. of entries in page table = No. of pages

No. of pages = $\frac{LAS}{\text{page size}}$

= $\frac{2^{26} \text{ byte}}{2^{10} \text{ byte}}$

= 2^{16}

PYQ-2022.

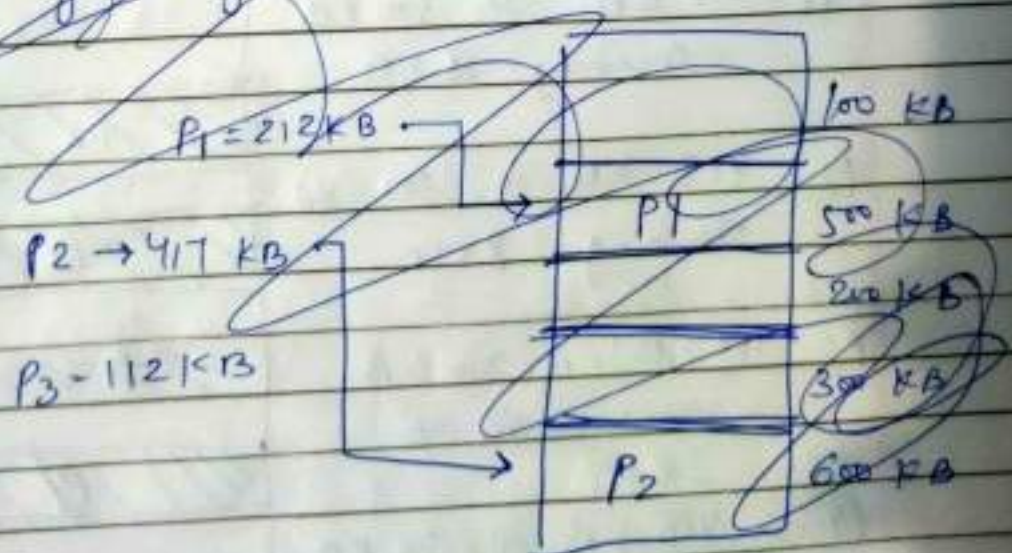
Q. Given five memory portion of 100 KB, 500 KB, 200 KB, 300 KB and 600 KB (in order). How would the first-fit, best-fit and worst-fit algorithm place processes ^{212, 417, 112 KB} (in order) which algorithm make the most efficient use of memory?

→ Given

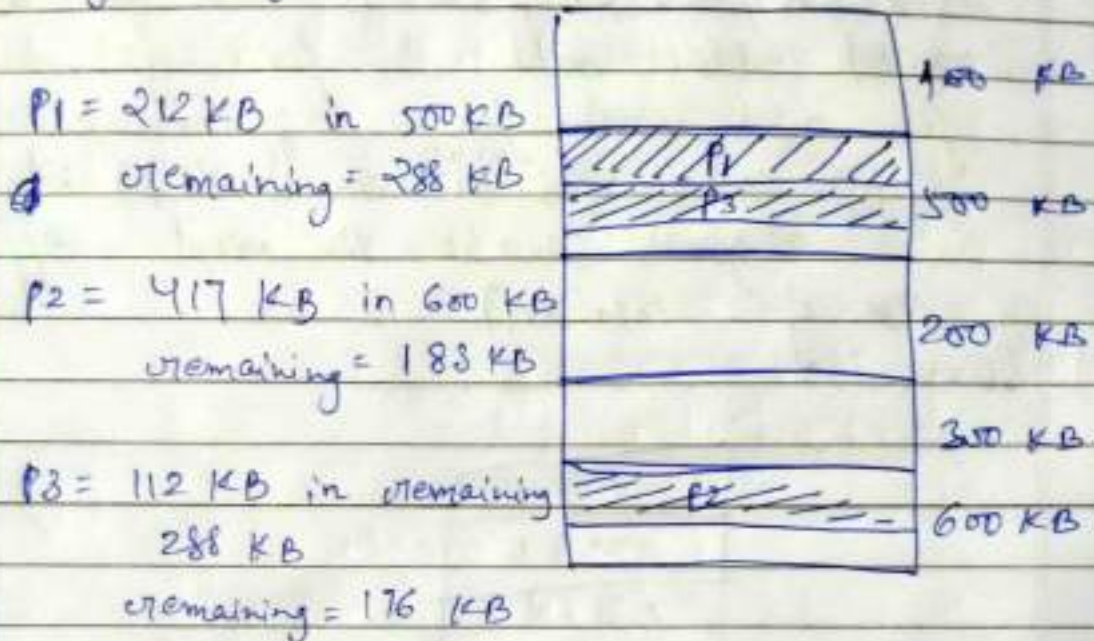
100 KB
500 KB
200 KB
300 KB
600 KB

Process: $P_1 = 212 \text{ KB}$, $P_2 = 417 \text{ KB}$, $P_3 = 112 \text{ KB}$, $P_4 = 426 \text{ KB}$

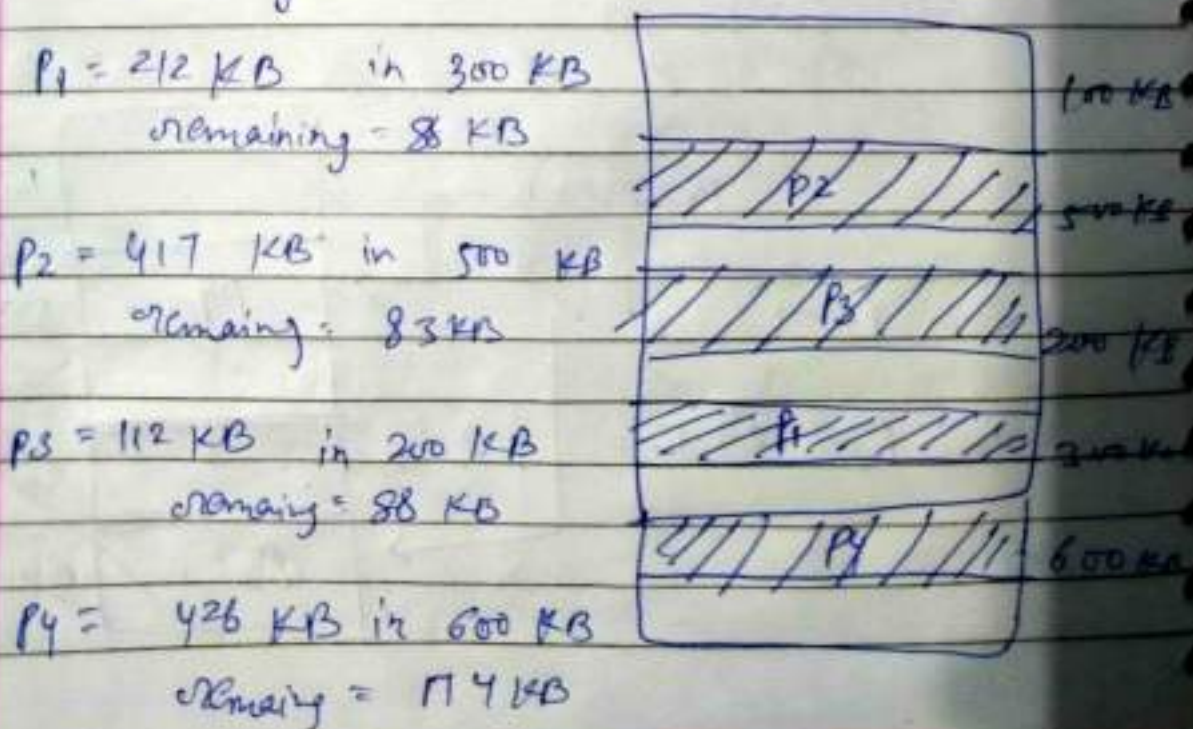
⊕ ~~first-fit~~



① first fit



② Best fit



② Worst fit

$P_1 = 212 \text{ KB}$ in 600 KB
 remaining = 388

$P_2 = 417 \text{ KB}$ in 500 KB
 remaining = 83 KB

$P_3 = 112 \text{ KB}$ in 600 KB
 remaining = 388
 remaining = 276 KB

$P_4 = 426 \text{ KB}$ (No allocation)



Q. 20

Virtual memory

Virtual memory = Technique that allows your system to use more memory than the RAM actually installed.

It does this by using secondary storage (SSD/HDD) as extra RAM.

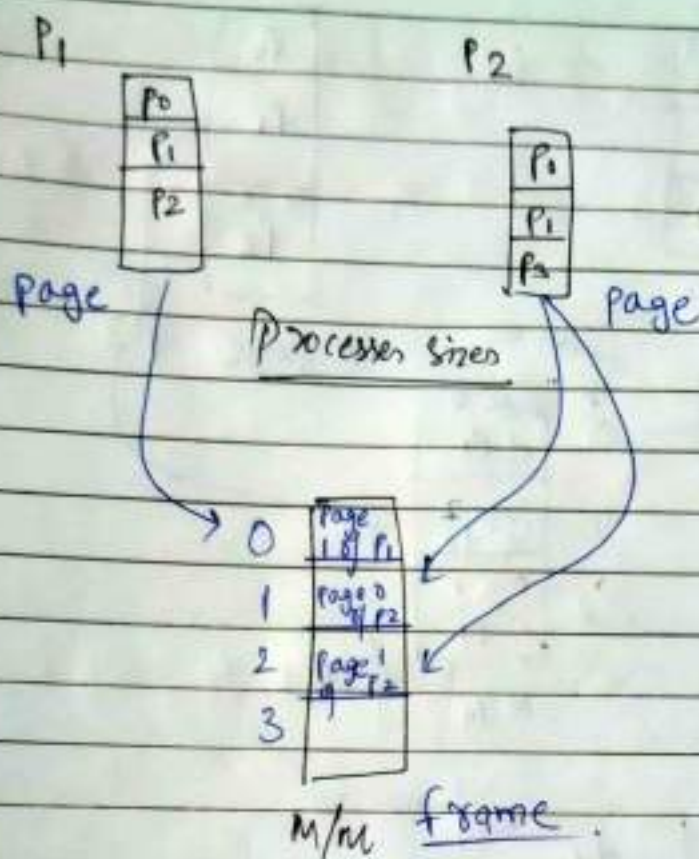
→ Why do we need virtual memory?

- Apps need more memory than available RAM.
- RAM is expensive & limited.
- Not all parts of a program are needed at the same time.

Virtual memory solves this by loading only required pages into RAM.

How virtual memory works :-

- 1) Program is divided into pages.
- 2) RAM is divided into frames.
- 3) Only the needed pages are kept in RAM.
- 4) Remaining pages are stored on disk (swap space / page file).
- 5) If a page needed → bring page from disk → place in RAM.
- 6) If RAM is full → remove an old page (page replacement).



Important terms:-

1. Demand paging :-

load a page only when its needed.

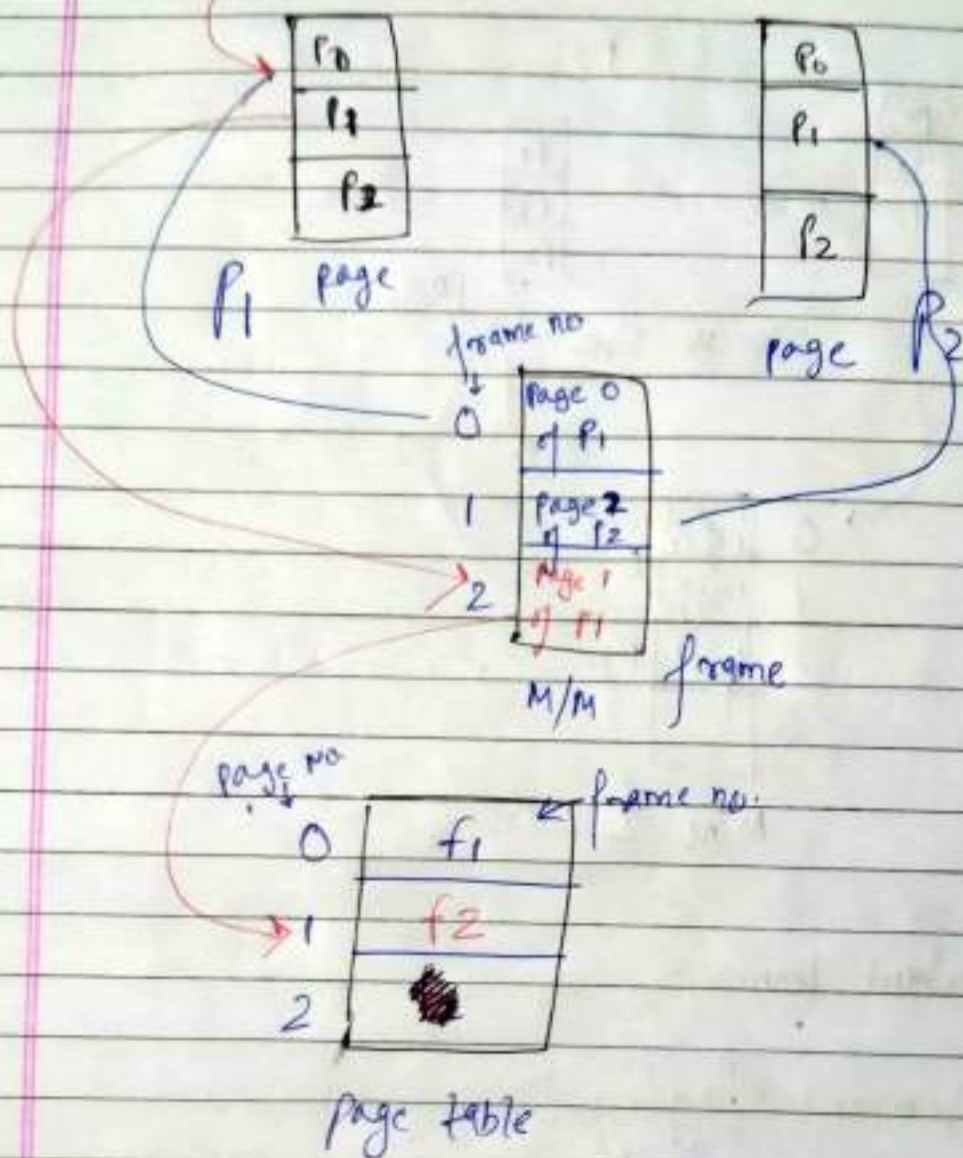
2. Page fault :

Occurs when :

- CPU request a page.
- Page is not in RAM (M/M).
- OS brings the page from disk $\rightarrow$ into an empty frame.

OS $P1 \rightarrow (\text{Page})$

Date: / /
Page:



CPU $\rightarrow$ P1 $\rightarrow$ page 1.

talking about P1 so page table is filled on only 0th place. 2 is empty because it contain ~~page~~ page 2 of P2 not P1.

• Can st.

• same $\rightarrow$ more page faults

Q.21. Example:- Consider page reference string 1, 3, 0, 3, 5, 6, 3 with 3-page frames. Find the number of page faults using FIFO page replacement Algorithm.

$\Rightarrow$

Page reference : 1, 3, 0, 3, 5, 6, 3.

	1	3	0	3	5	6	3
frame 3			0	0	0	0	3
frame 2		3	3	3	3	6	6
frame 1	1	1	1	1	5	5	5
	Miss	Miss	Miss	Hit	Miss	Miss	Miss

total page fault = 6
total page hit = 1

- Initially, all slots are empty, so when 1, 3, 0 come they are all allocated to the empty slots $\rightarrow$ 3 page faults.
- When 3 comes, it is already in memory so $\rightarrow$ 0 page fault.
- Then 5 comes, it is not available in memory, so it replaces the oldest page slot i.e. 1 $\rightarrow$ 1 page fault.

• 6 comes, it is also not available in memory,
so it replaces the oldest page slot i.e. 3
→ 1 page fault.

• finally, when 3 come it is not available,
so it replace 1 → ~~1~~ 1 page fault.

Q.22 Eg; Consider the page reference
 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with 4-page
 frame. Find no. of page fault using
 optimal page replacement algorithm.

⇒ Page reference:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3

	7	0	1	2	0	3	0	4	2	3	0	3	2	3
frame 4				2	2	2	2	2	2	2	2	2	2	2
frame 3			1	1	1	1	1	4	4	4	4	4	4	4
frame 2		0	0	0	0	0	0	0	0	0	0	0	0	0
frame 1	7	7	7	7	7	3	3	3	3	3	3	3	3	3
	Miss	Miss	Miss	Miss	Hit	Miss	Hit	Miss	Hit	Hit	Hit	Hit	Hit	Hit

Total page fault = 6.
Total hit = 8.

Initially, all slots are empty, so when 7, 0, 1, 2 are allocated to the empty slots ~~and~~ slots $\rightarrow$ 4 page fault.

- 0 is already there so $\rightarrow$ 0 page fault. or 1
- When 3 came it will take the place of 7, because it is not used for the longest duration of time in future $\rightarrow$ 1 page fault.
- 0 is already there so $\rightarrow$ 0 page fault.
- 4 will take place of 1 $\rightarrow$ 1 page fault.
- For the further page referencing $\rightarrow$ 0 page fault.

• Key points:-

- OPT requires future knowledge $\rightarrow$ not implement in real OS.
- Used only in theory.
- OPT never show belady's anomaly (FIFO sometimes show).

Eg: Consider the page reference string
 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with 4-page
 frame. Find number of page faults
 using LRU page replacement
 algorithm.

Page reference:-

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3.

	7	0	1	2	0	3	0	4	2	3	0	3	
frame 4				2	2	2	2	2	2	2	2	2	2
frame 3			1	1	1	1	4	4	4	4	4	4	4
frame 2		0	0	0	0	0	0	0	0	0	0	0	0
frame 1	7	7	7	7	7	3	3	3	3	3	3	3	3
	Miss	miss	miss	hit	miss	hit	miss	hit	hit	hit	hit	hit	hit

Total page fault = 6

Total hit = 8

- Initially, all slots are empty, so when 7, 0, 1, 2 are allocated to the empty slots $\rightarrow$ 4 page fault.
- 0 is already there so $\rightarrow$ 0 page fault.
- When 3 came it will take the place of 7 because it is least recently used $\rightarrow$ 1 page fault.
- 0 is already in memory so $\rightarrow$ 0 page fault.
- 4 will take place of 1 $\rightarrow$ 1 page fault.
- Now for the further page reference string $\rightarrow$ 0 page fault.

Q.24

Eg;

Page reference:- 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3

No. of frame = 4.

	7	0	1	2	0	3	0	4	2	3	0	3	2	3
frame 4				2	2	2	2	2	2	3	3	3	2	3
frame 3			1	1	1	1	1	1	1	1	1	1	1	1
frame 2		0	0	0	0	0	0	4	4	4	4	4	4	4
frame 1	7	7	7	7	7	7	7	7	7	7	7	7	7	7
	Miss	Miss	Miss	Hit	Hit	Hit	Hit	Hit	Hit	Hit	Miss	Miss	Miss	Miss

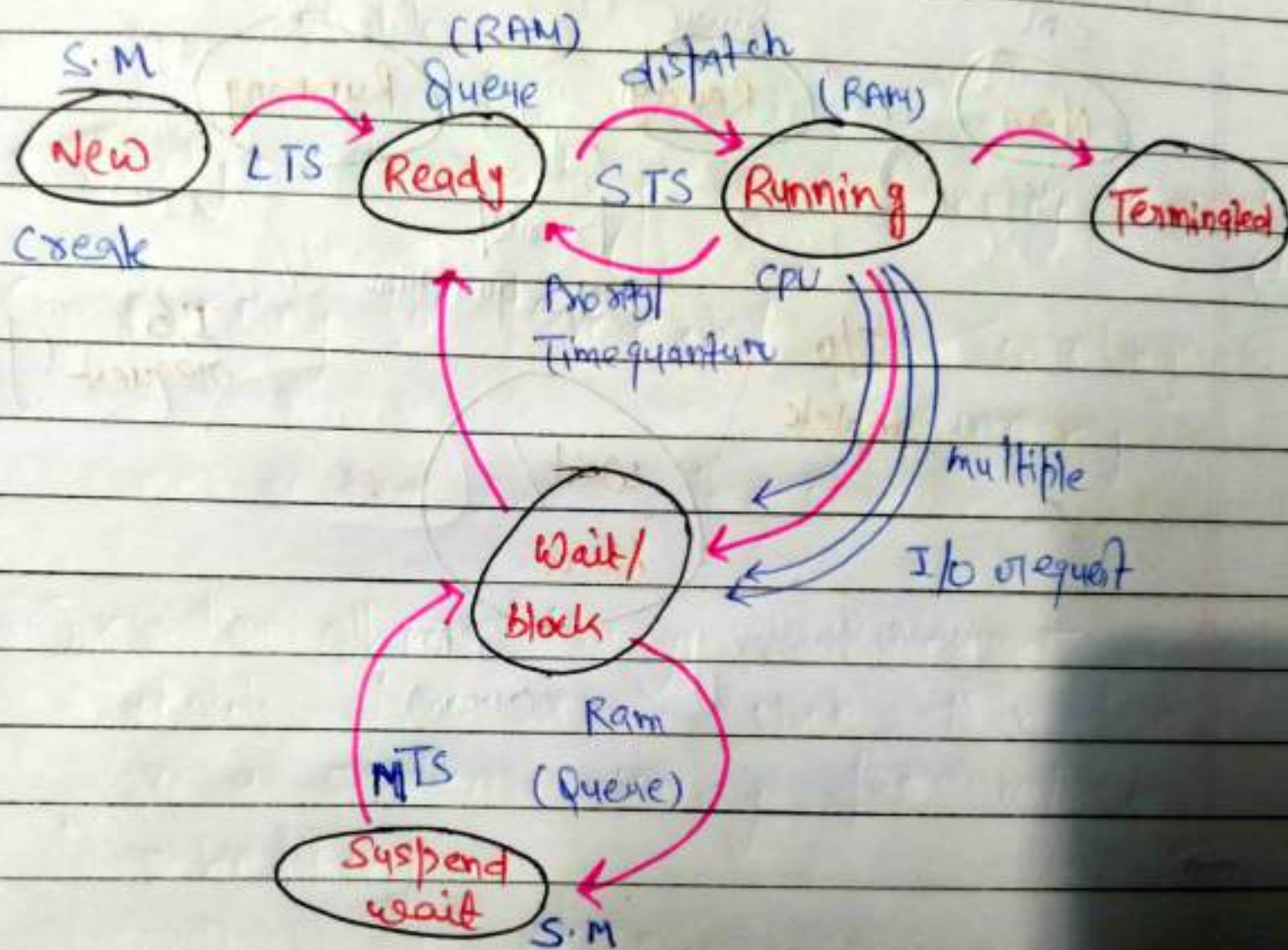
Total page fault = 12

TLB hit = 2.

Initially all slots are empty, so when 7, 0, 1, 2 are allocated to the empty slot $\rightarrow$ 4 page fault

- 0 is already there so $\rightarrow$ 0 page fault
- when 3 came it will take place of 0 because it is most recently used $\rightarrow$ 1 page fault.
- when 0 comes it will take place of 3 $\rightarrow$ 1 page fault.
- when 4 comes it will take place of 0 $\rightarrow$ 1 page fault.
- 2 is already in memory so $\rightarrow$ 0 page fault.
- ~~Similarly~~ when 3 comes it will take place of 2 $\rightarrow$ 1 page fault.
- Similarly - - -

Q.25.



→ Multiple I/O request sent so wait/block it is full filled (because it is in RAM).

LTS → Long-term scheduler → decides which jobs from the secondary storage (secondary memory) will be admitted into the system (main/primary memory).

Role → degree of multiprogramming

Non-preemptive → Once a process starts executive on the CPU, it cannot be interrupt until it finishes execution.

- No context-switched overhead (no change of state in backward).

RAM).

MTS → Medium term scheduler → Decided which process should be suspended (swapped out to disk (S.M)) or resumed (brought back into memory)

Role → Temporarily reduce the degree of multiprogramming when system is overloaded.

STS → Short term scheduler → Decide which process from ready queue get CPU next (running).

Role → Allocated CPU to processes.

Akash